DATA STRUCTURES & ALGORITHMS

CODING TEST — LINKED LIST, STACK & QUEUE

Name: _____	**Roll No.:** _____
Class/Batch: _____	**Date:** _____

Instructions

• Attempt all 6 questions. Each question carries **10 marks** (Total: 60 marks).

• Write a complete, working solution in Java. Include necessary classes and methods.

• Do not use built-in data structures to replace the structure being tested. For example, implement a stack using your own linked nodes when asked.

• Handle edge cases such as empty structures, single-element inputs, and invalid operations.

• Mention time and space complexity. Explain your approach briefly where required.

SECTION A — LINKED LIST (20 marks)

Q1. Reverse Nodes in Groups of K

Given the head of a singly linked list and an integer **k**, reverse the nodes in groups of k. If the number of nodes left is less than k, leave those nodes in their original order. You must change links only; do not swap node values and do not use extra arrays/lists.
Example: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8, k = 3
Output: 3 → 2 → 1 → 6 → 5 → 4 → 7 → 8
Task: Implement the solution and state its time and auxiliary space complexity.

Q2. Detect and Remove a Cycle

A singly linked list may contain a cycle. Implement a method that detects whether a cycle exists and, if it does, removes the cycle without losing any node. Return whether a cycle was found. You must use O(1) extra space and must not use a HashSet.
Task: Explain how you locate the start of the cycle, how you break it, and analyze complexity.

SECTION B — STACK (20 marks)

Q3. Design a Min Stack

Design a stack supporting **push(x)**, **pop()**, **top()**, and **getMin()**. **getMin()** must return the minimum element currently in the stack. All operations must run in O(1) time. Do not use any built-in stack class.
Task: Implement the data structure and safely handle calls on an empty stack. Explain how duplicate minimum values are handled.

Q4. Evaluate an Expression in Postfix Notation

Given a valid postfix expression containing non-negative integers and the operators +, –, ×, and ÷, evaluate it using a stack. Tokens are separated by spaces, and integer division truncates toward zero. For example, **"12 3 4 * + 2 /"** evaluates to **12**.
Task: Implement the evaluator without using built-in stack classes. Describe how operand order is preserved for subtraction and division, and handle division by zero.

SECTION C — QUEUE (20 marks)

Q5. Implement a Queue Using Two Stacks

Implement a FIFO queue with **enqueue(x)**, **dequeue()**, **peek()**, and **isEmpty()** using exactly two stacks and no queue library. Each operation should have amortized O(1) time complexity.
Task: Provide complete Java code, handle empty-queue operations, and explain why each element is moved between the stacks at most a constant number of times.

Q6. Circular Queue Using an Array

Implement a fixed-capacity circular queue using an array. Support **offer(x)**, **poll()**, **front()**, **rear()**, **isEmpty()**, and **isFull()**. All operations must run in O(1) time. Do not shift array elements after removal.
Task: Correctly manage wrap-around, full/empty states, and capacity. Explain the role of the front/rear indices or a size variable.

Submission checklist: Code compiles • Edge cases tested • Complexity stated